GPU architecture, Matrix Multiplication, and Triton

Danyang Zhuo

Duke University

Some materials are from Tianqi Chen's and Hao Zhang's slides

Logistics

- If you have not started on programming assignment, please do so immediately.

- The course is already full, and the waitlist has many students.
  - If you ABSOLUTELY wants to take the course, please send me an email.

“Research” project

- Core component of the course. Form team of 1-3 students.

- What’s the goal of a “research” project?

“Research” project

- Core component of the course. Form team of 1-3 students.

- What’s the goal of a “research” project?
  - Study a focused area
  - Writing non-trivial code to demonstrate your understanding
  - Write project reports

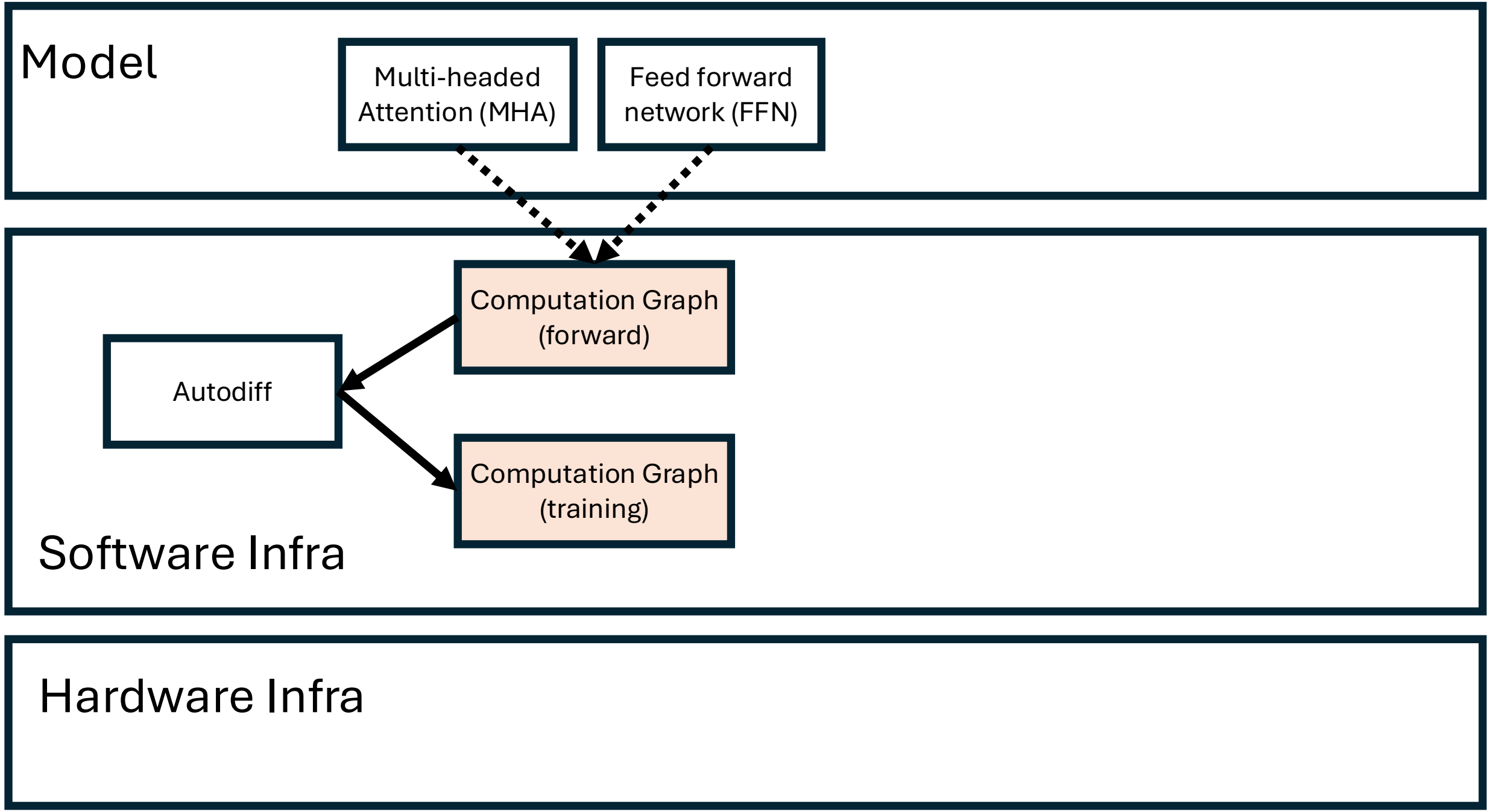
If you cannot find a project idea

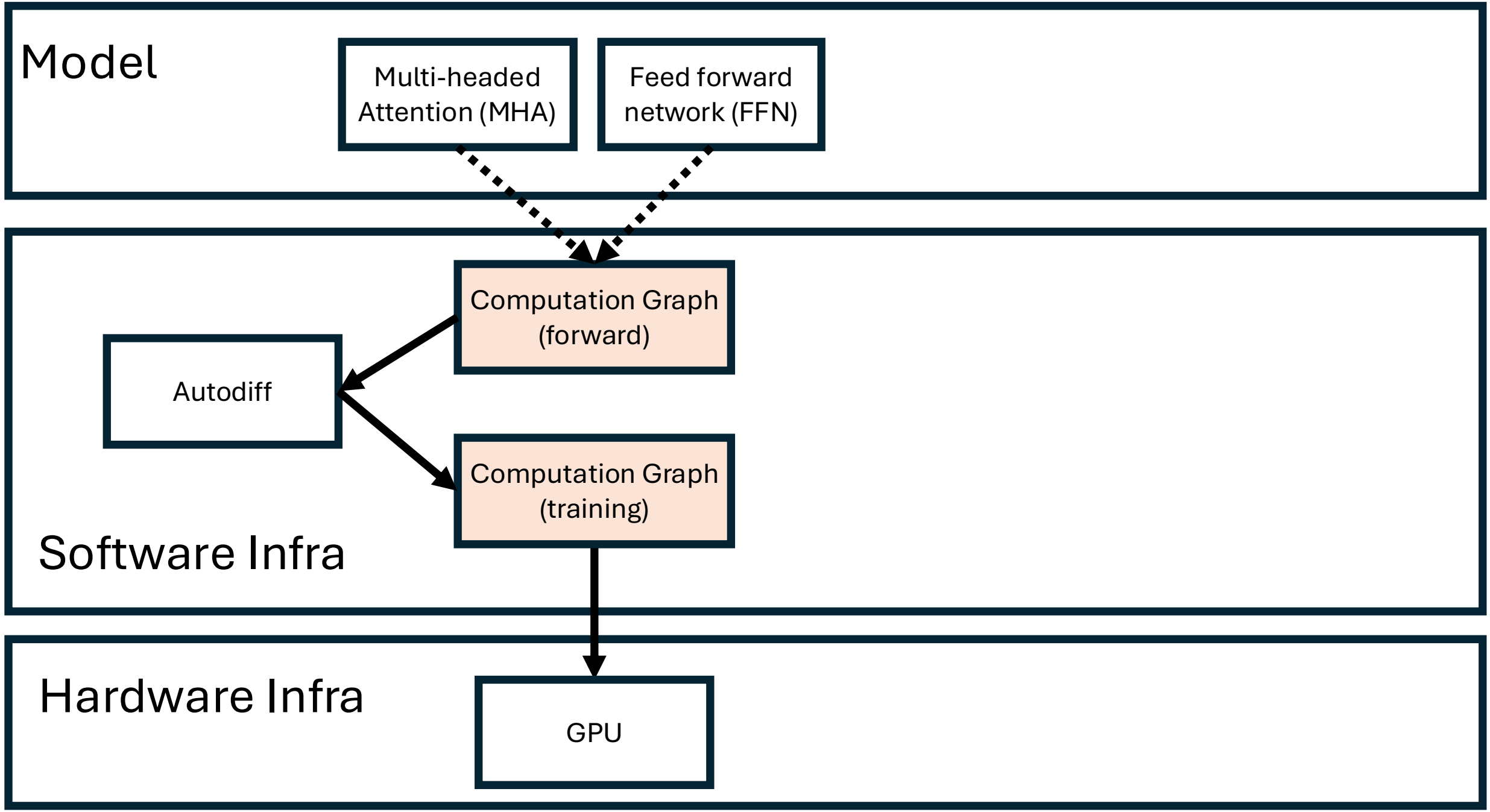
- If you are a PhD student, try to think about how to combine your existing research
  - Hardware -> GPU kernel design, quantization
  - Database -> Vector DB
  - Compiler, formal methods -> PyTorch, TVM, graph optimization
  - System -> distributed training, efficient inference
  - Networking -> collective communication
  - ML, algorithm -> sparsity, speculative decoding

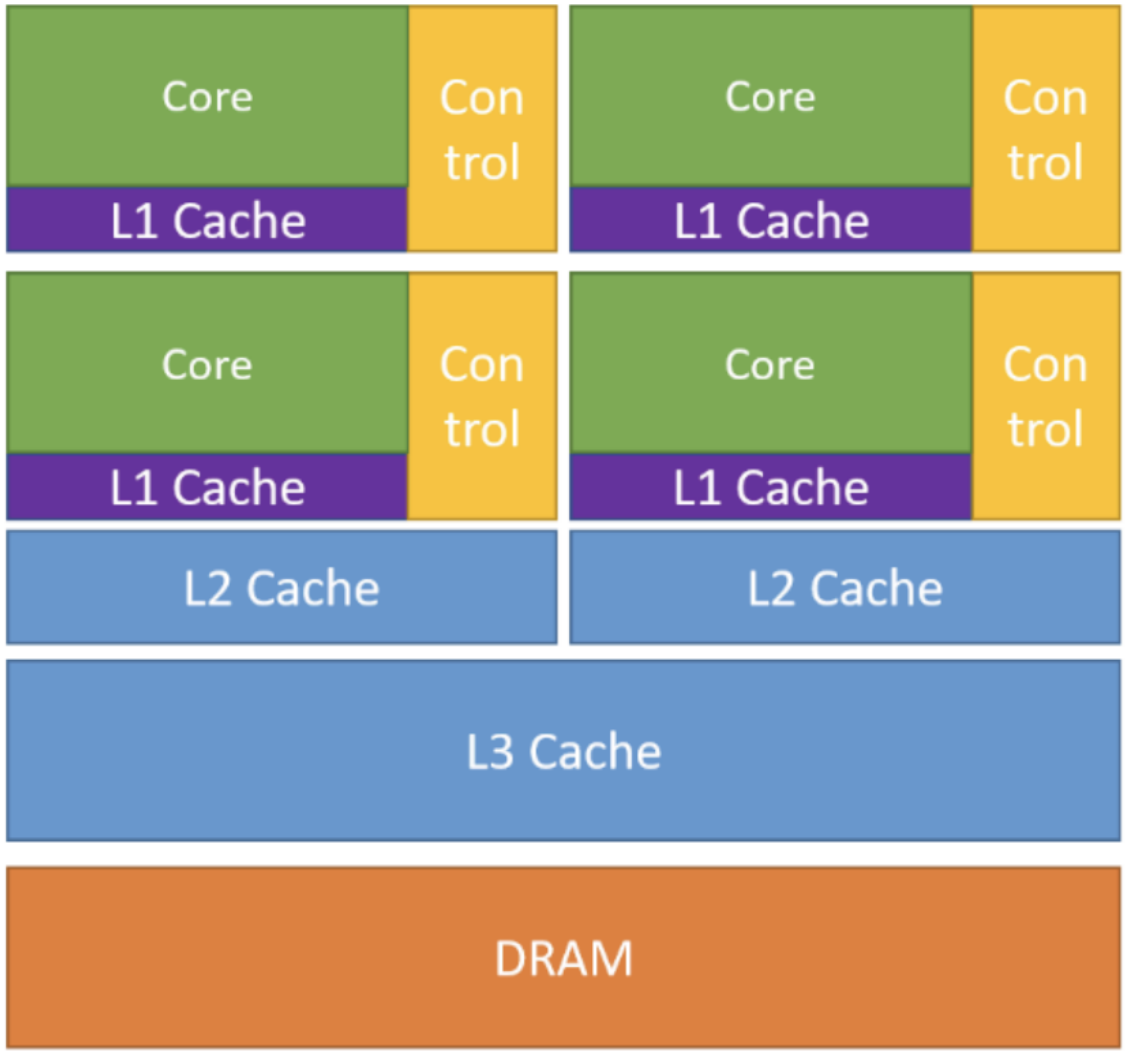
If you cannot find a project idea

- I will present a list of project ideas for you to choose, but you still need to decide on your team.

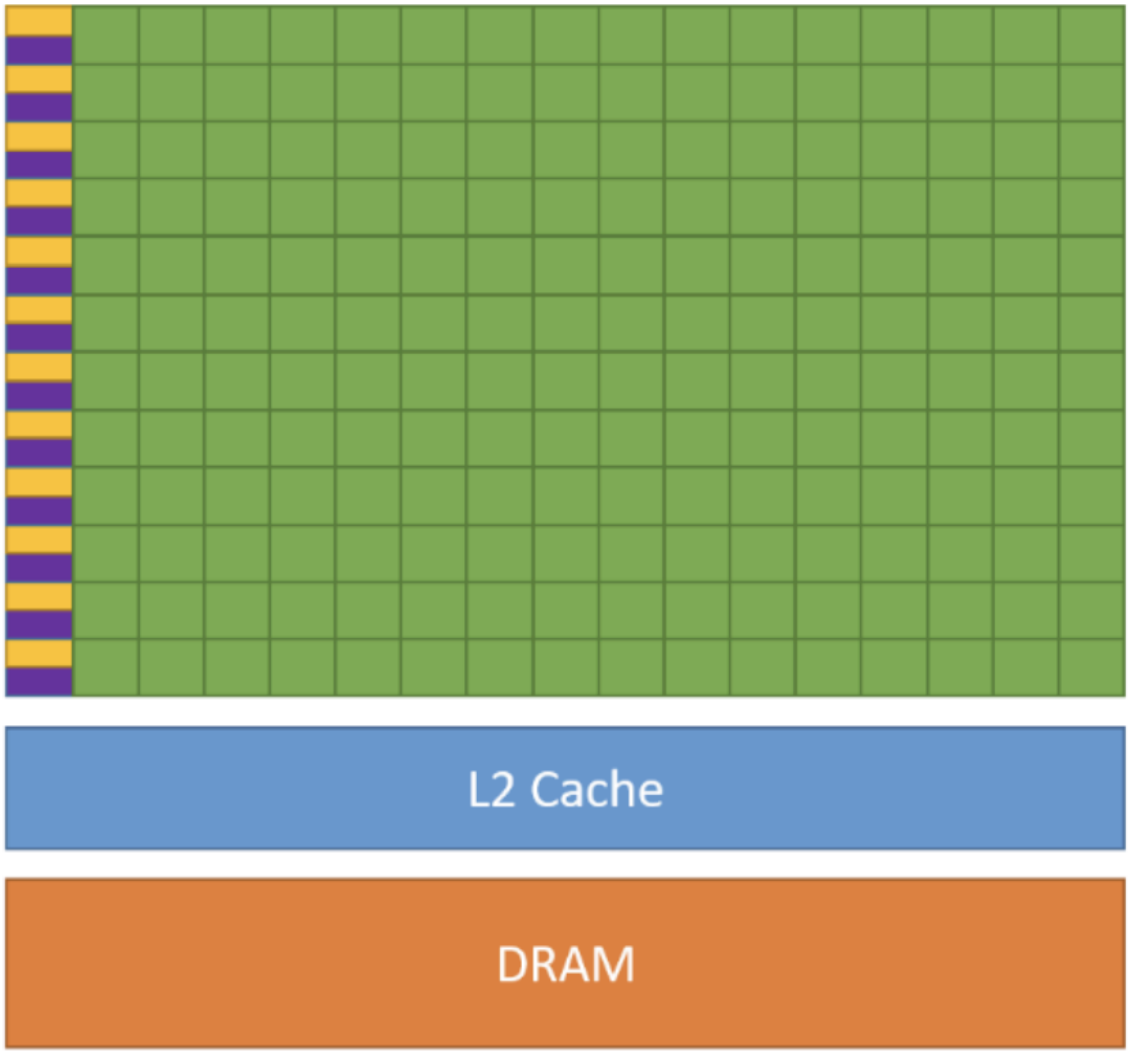
- 1-person team: survey of a focused area, running benchmark code, understanding performance
- 2- or 3-person team: non-trivial software development in the space of ML systems
  - Reproducing some known paper
  - Developing new feature on known projects


Model
Multi-headed Attention (MHA)
Feed forward network (FFN)
Computation Graph (forward)
Autodiff
Computation Graph (training)
Software Infra
Hardware Infra


Model
Multi-headed Attention (MHA)
Feed forward network (FFN)
Computation Graph (forward)
Autodiff
Computation Graph (training)
Software Infra
Hardware Infra
GPU

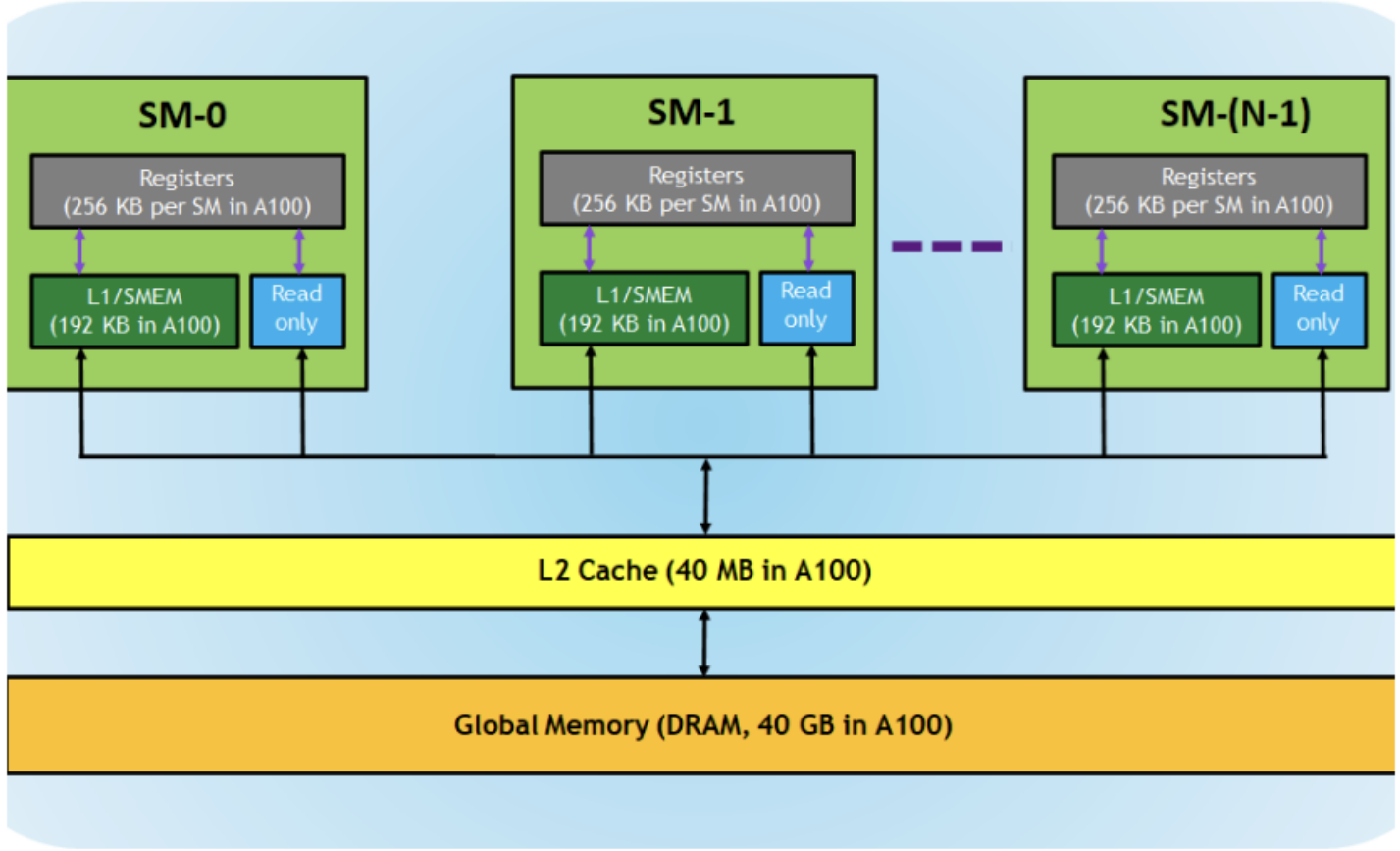

Core
Con
trol
L1 Cache
Core
Con
trol
L1 Cache
Core
Con
trol
L1 Cache
Core
Con
trol
L1 Cache
L2 Cache
L2 Cache
L3 Cache
DRAM

CPU


L2 Cache
DRAM

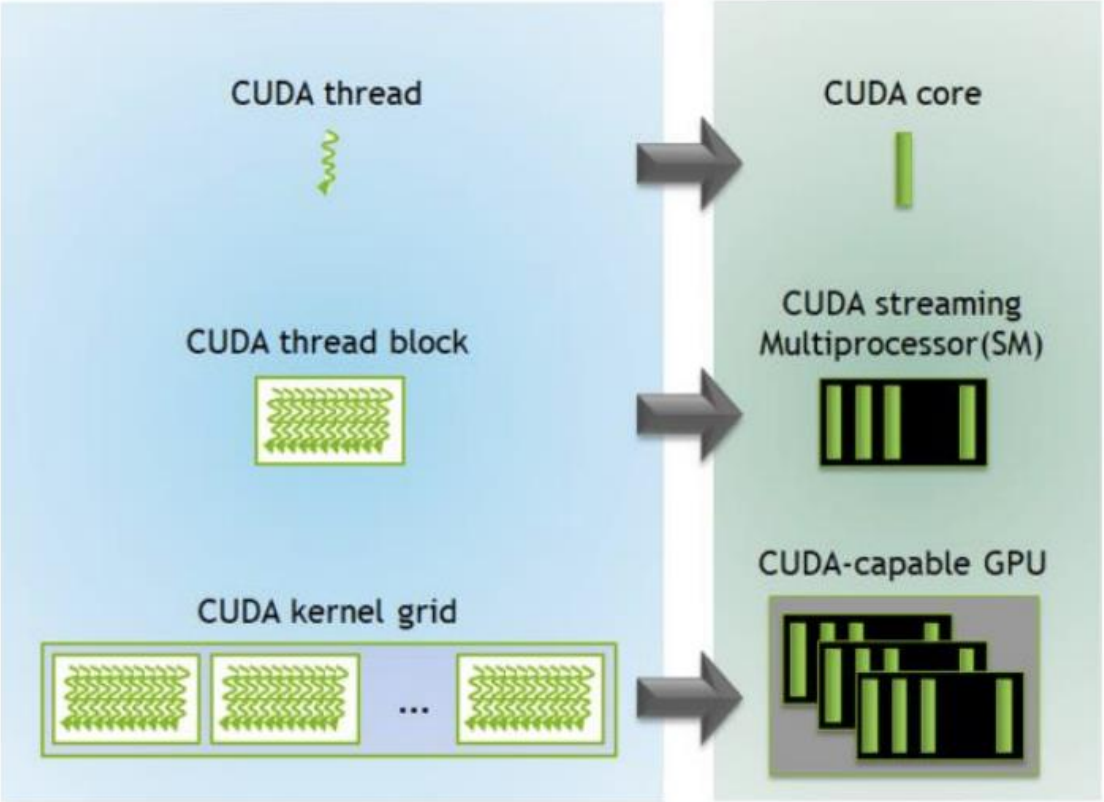
GPU

GPU Overview



Kernel, Threads, Blocks, Grids

- Threads: smallest units to process a chunk of data
- Blocks: A group of threads that share memory
- Grid: A collection of blocks that execute the same kernel
- Kernel: CUDA program executed by many CUDA cores in parallel



An Example CUDA Program: Matrix Add

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x,
               Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAddDoubleB<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

- "launch a grid of CUDA thread
- blocks" Call returns when all threads have terminated

```
__device__ float doubleValue(float x)
{
   return 2 * x;
}

// kernel definition
__global__ void matrixAddDoubleB(float A[Ny][Nx],
                                 float B[Ny][Nx],
                                 float C[Ny][Nx])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;

   C[j][i] = A[j][i] + doubleValue(B[j][i]);
}
```

- __global__ denotes a CUDA kernel function runs on GPU
- Each thread indexes its data using blockIdx, blockDim, threadIdx and execute the compute

Separation CPU and GPU Execution

```
const int Nx = 12;
const int Ny = 6;

dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x,
               Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays

// this call will cause execution of 72 threads
// 6 blocks of 12 threads each
matrixAddDoubleB<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

- Host code: serial execution on CPU

```
__device__ float doubleValue(float x)
{
   return 2 * x;
}

// kernel definition
__global__ void matrixAddDoubleB(float A[Ny][Nx],
                                 float B[Ny][Nx],
                                 float C[Ny][Nx])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;

   C[j][i] = A[j][i] + doubleValue(B[j][i]);
}
```

- Device code: SIMD parallel execution on GPUs

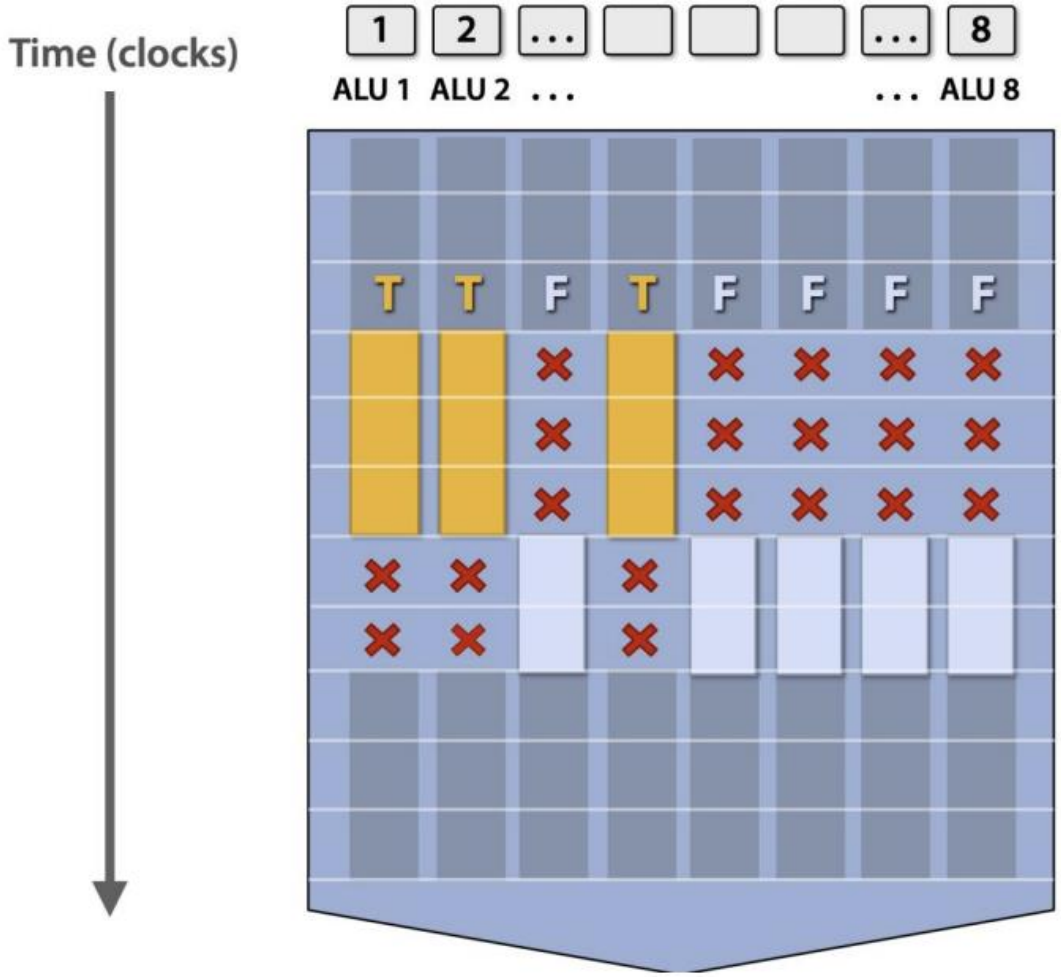
SIMD Constraints: how to handle control flow?

SIMD requires all ALUs/Core Must proceed in the same pace

- Why?
- Let's look at a control flow example

```
// kernel definition
__global__ void f(float A[N])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   float x = A[i];
   if (x > 0) {
      x = 2.0f * x;
   } else {
      x = exp(x, 5.0f);
   }
   A[i] = x;
}
```

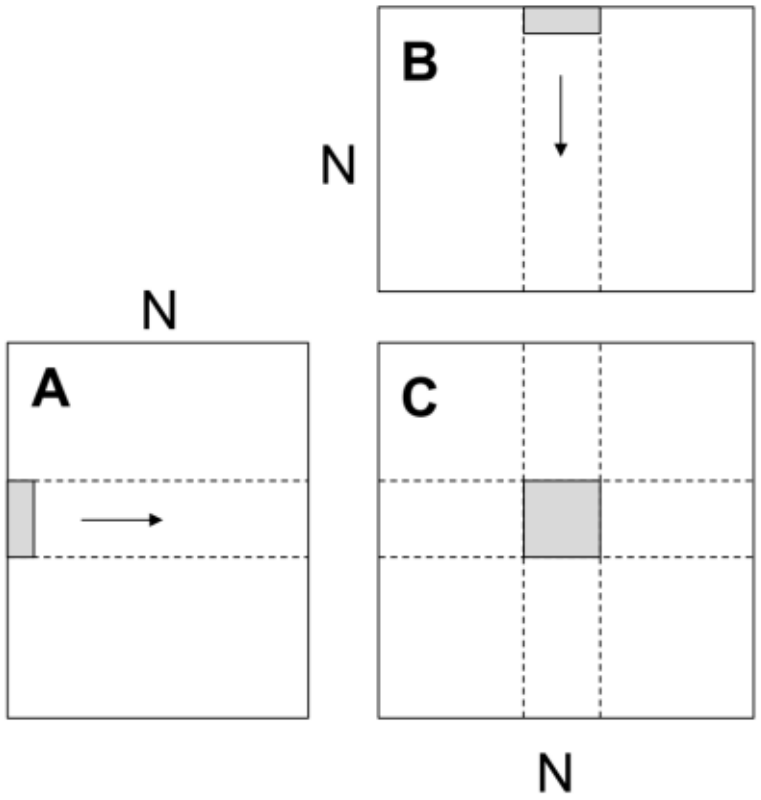
Handling Control Flow




```
// kernel definition
__global__ void f(float A[N])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   float x = A[i];
   if (x > 0) {
      x = 2.0f * x;
   } else {
      x = exp(x, 5.0f);
   }
   A[i] = x;
}
```

Case study: GPU Matmul v1

- C = A x B
- Q: what's the work that can be parallelizec
- 💡Each thread computes one element!

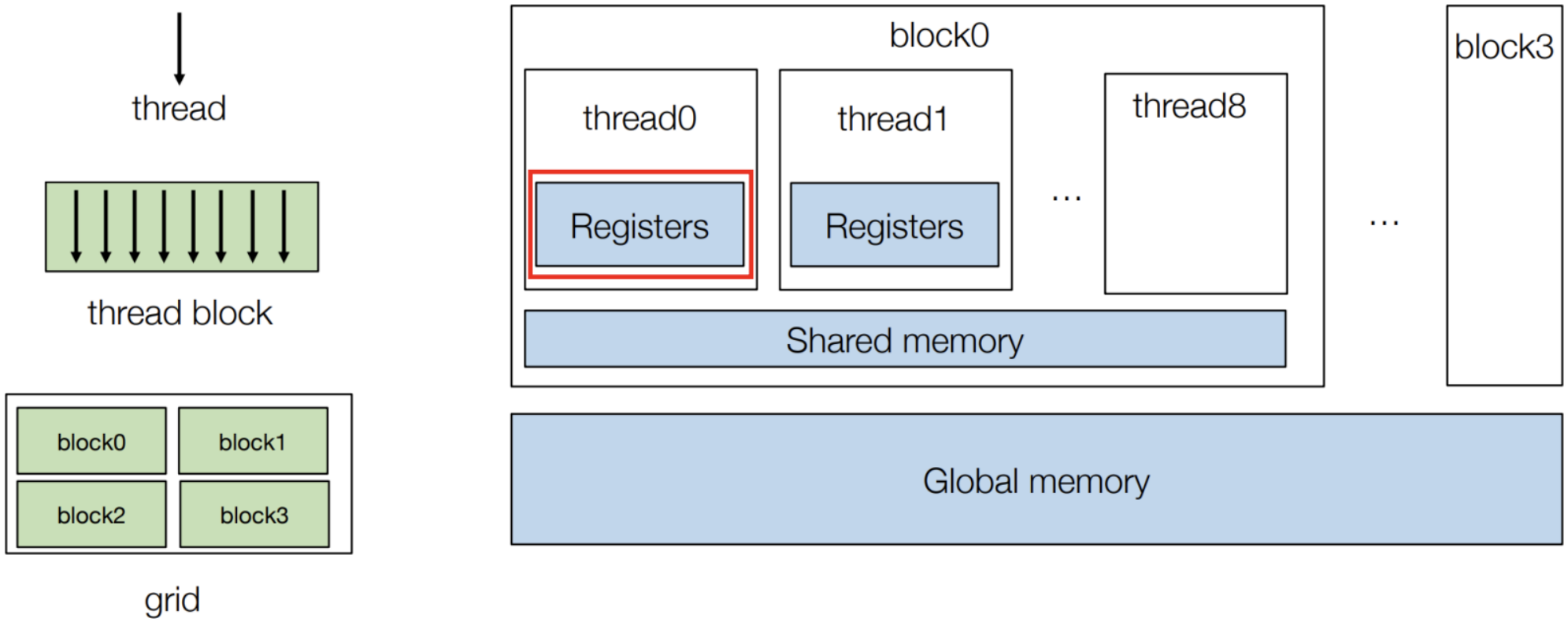



```
int N = 1024;
dim3 threadsPerBlock(32, 32, 1);
dim3 numBlocks(N/32, N/32, 1);

matmul<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

```
__global__ void mm(float A[N][N], float B[N][N], float C[N][N]) {
  int x = blockIdx.x * blockDim.x + threadIdx.x;
  int y = blockIdx.y * blockDim.y + threadIdx.y;

  result = 0;
  for (int k = 0; k < N; ++k) {
    result += A[x][k] * B[k][y];
  }
  C[x][y] = result;
}
```

- Global memory read per thread?
  - N + N = 2N
- # threads?
  - N^2
- Total global memory access?
  - N^2 * 2N = 2N^3
- Memory?
  - 1 float per thread

Recall Memory Hierarchy and Register tiling

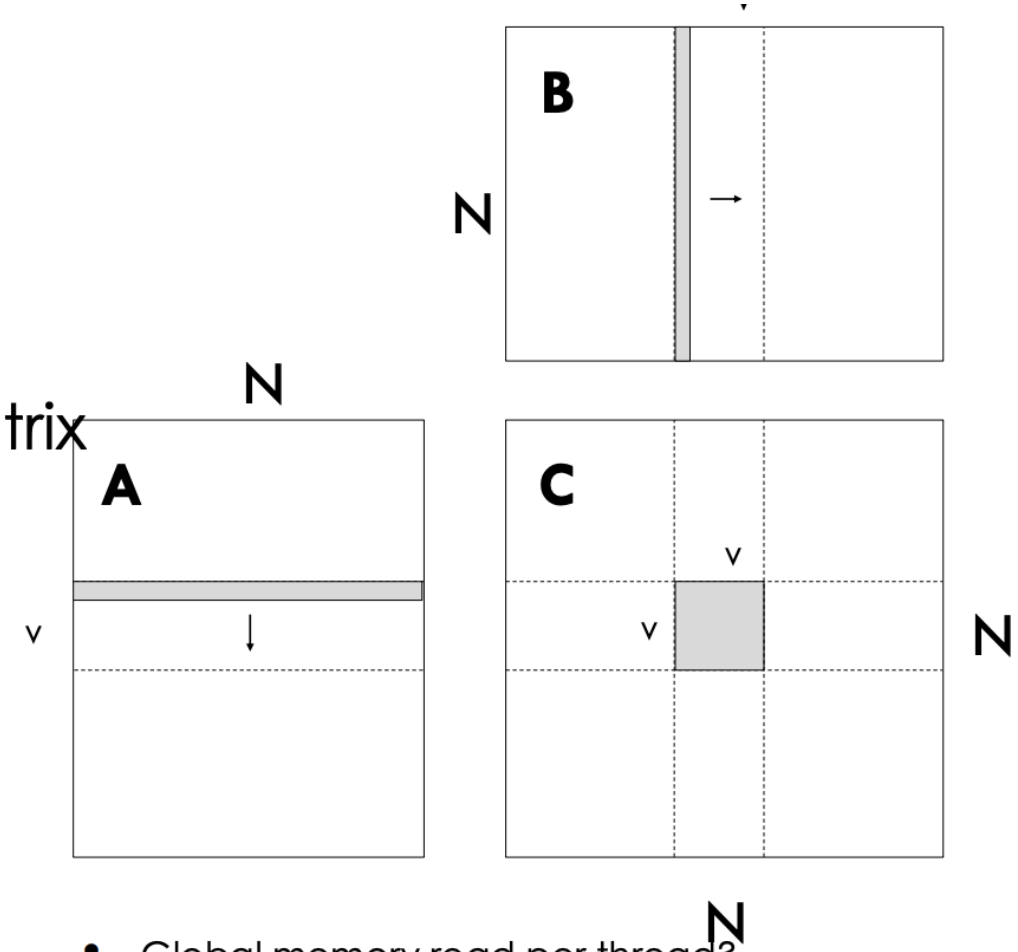


💡Each thread uses more thread-level registers to compute outputs to save I/o

GPU Matmul v1.5: Thread Tiling

- Each thread computes a VxV submatrix

```
__global__ void mm(float A[N][N], float B[N][N], float C[N][N]) {
  int ybase = blockIdx.y * blockDim.y + threadIdx.y;
  int xbase = blockIdx.x * blockDim.x + threadIdx.x;

  float c[V][V] = {0};
  float a[N], b[N];
  for (int x = 0; x < V; ++x) {
    a[:] = A[xbase * V + x, :];
    for (int y = 0; y < V; ++y) {
      b[:] = B[:, ybase * V + y]
      for (int k = 0; k < N; ++k)
        c[x][y] += a[k] * b[k];
      }
    }
  }
  C[xbase * V: xbase*V + V, ybase * V: ybase*V + V] = c[:];
}
```

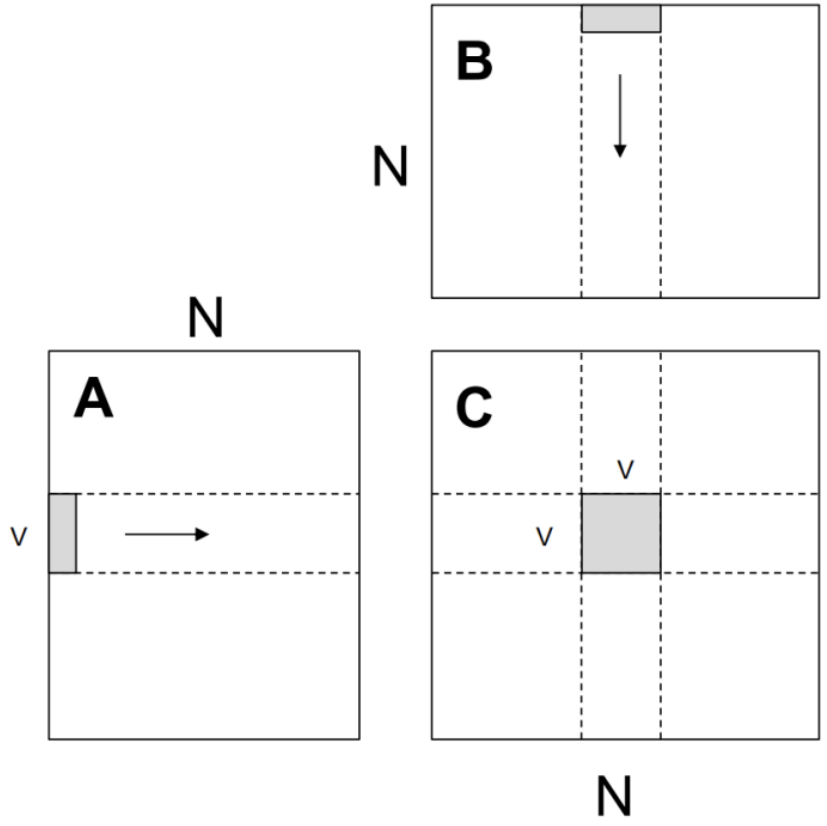


- Global memory read per thread?
  - NV + NV^2
- # threads?
  - N/V * N/V = N^2/V^2
- Total global memory access?
  - N^2 / V^2 * (NV + NV^2) = N^3/V + N^3
- Memory?
  - V^2 + 2N float per thread

GPU Matmul v2: Can we do better?

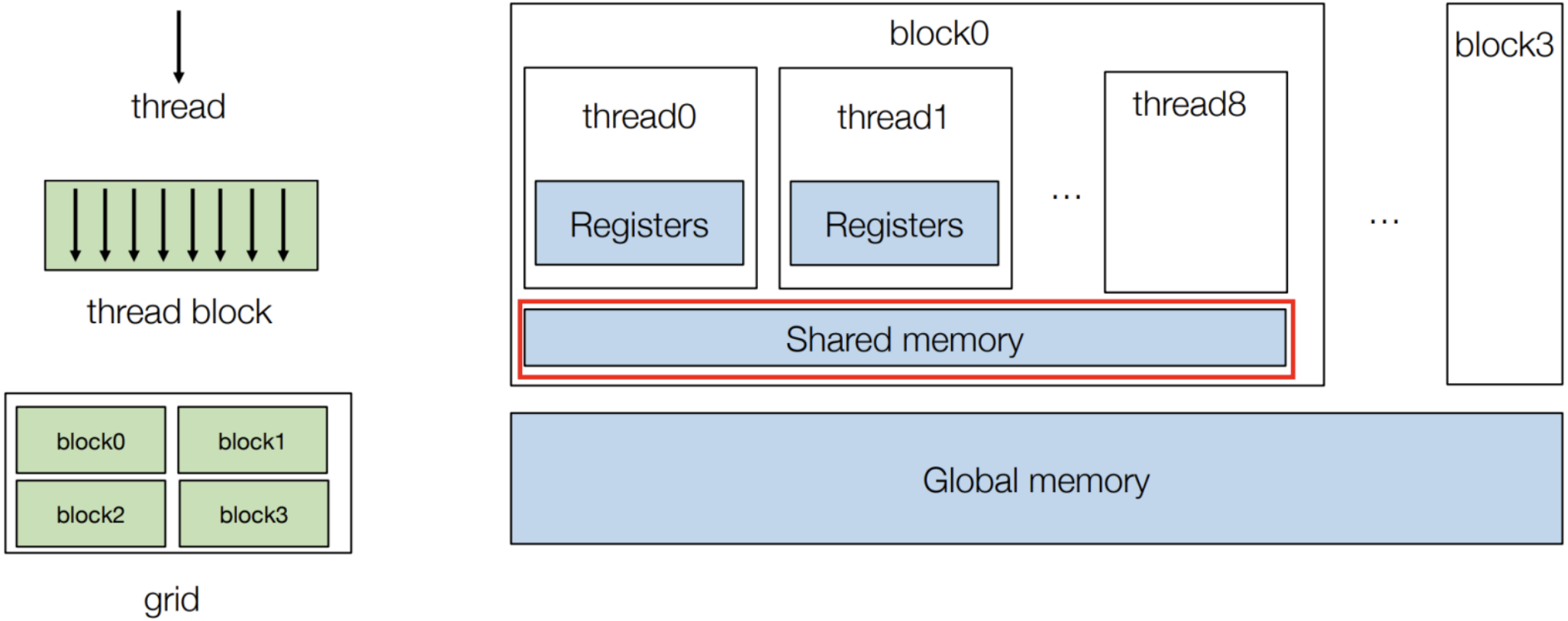
- Each thread computes a VxV submatrix
- 💡compute partial sum: $[X_1, X_2]\begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} = X_1Y_1 + X_2Y_2$

```
__global__ void mm(float A[N][N], float B[N][N], float C[N][N]) {
  int ybase = blockIdx.y * blockDim.y + threadIdx.y;
  int xbase = blockIdx.x * blockDim.x + threadIdx.x;

  float c[V][V] = {0};
  float a[V], b[V];
  for (int k = 0; k < N; ++k) {
    a[:] = A[xbase*V : xbase*V + V, k];
    b[:] = B[k, ybase*V : ybase*V + V];
    for (int y = 0; y < V; ++y) {
      for (int x = 0; x < V; ++x) {
        c[x][y] += a[x] * b[y];
      }
    }
  }
  C[xbase * V : xbase*V + V, ybase*V : ybase*V + V] = c[:];
}
```



- Global memory read per thread?
  - NV * 2
- # threads?
  - N/V * N/V = N^2/V^2
- Total global memory access?
  - N^2 / V^2 * 2NV = 2N^3/V
- Memory?
  - V^2 + 2V float per thread

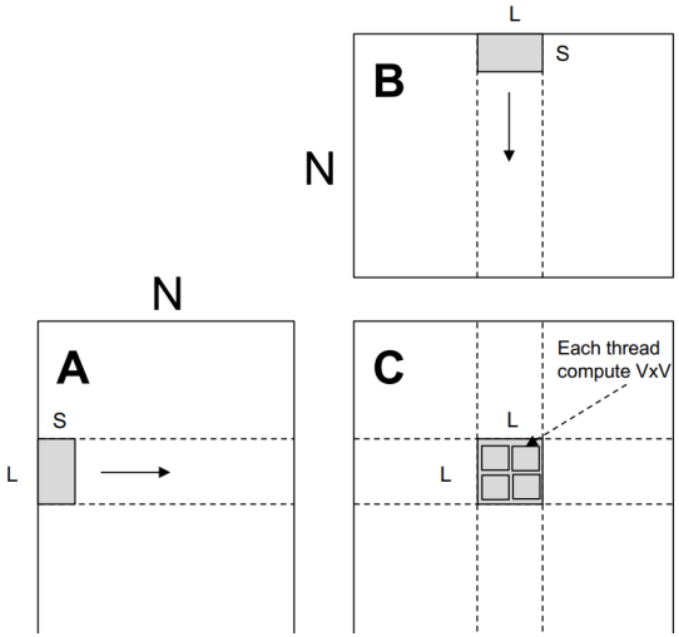
Recall Memory Hierarchy and Cache tiling



💡Try to utilize block-level shared memory (SRAM)

GPU Matmul v3: SRAM Tiling (GPU)

- Use block shared mem
- A block computes a L x L submatrix
- Then a thread computes a V x V submatrix and reuses the matrices in shared block memory




```
__global__ void mm(float A[N][N], float B[N][N], float C[N][N]) {
  __shared__ float sA[S][L], sB[S][L];
  float c[V][V] = {0};
  float a[V], b[V];
  int yblock = blockIdx.y;
  int xblock = blockIdx.x;

  for (int ko = 0; ko < N; ko += S) {
    __syncthreads();
    // needs to be implemented by thread cooperative fetching
    sA[:, :] = A[ko : ko + S, yblock * L : yblock * L + L];
    sB[:, :] = B[ko : ko + S, xblock * L : xblock * L + L];
    __syncthreads();
    for (int ki = 0; ki < S; ++ ki) {
      a[:] = sA[ki, threadIdx.y * V : threadIdx.y * V + V];
      b[:] = sA[ki, threadIdx.x * V : threadIdx.x * V + V];
      for (int y = 0; y < V; ++y) {
        for (int x = 0; x < V; ++x) {
          c[y][x] += a[y] * b[x];
        }
      }
    }
  }
  int ybase = blockIdx.y * blockDim.y + threadIdx.y;
  int xbase = blockIdx.x * blockDim.x + threadIdx.x;
  C[ybase * V : ybase*V + V, xbase*V : xbase*V + V] = c[:];
}
```

Memory overhead?

- Global memory access per threadblock
  - 2LN
- Number of threadblocks:
  - N^2 / L^2
- Total global memory access:
  - 2N^3 / L
- Shared memory access per thread:
  - 2VN
- Number of threads
  - N^2 / V^2
- Total shared memory access:
  - 2N^3 / V

```
__global__ void mm(float A[N][N], float B[N][N], float C[N][N]) {
  __shared__ float sA[S][L], sB[S][L];
  float c[V][V] = {0};
  float a[V], b[V];
  int yblock = blockIdx.y;
  int xblock = blockIdx.x;

  for (int ko = 0; ko < N; ko += S) {
    __syncthreads();
    // needs to be implemented by thread cooperative fetching
    sA[:, :] = A[ko : ko + S, yblock * L : yblock * L + L];
    sB[:, :] = B[ko : ko + S, xblock * L : xblock * L + L];
    __syncthreads();
    for (int ki = 0; ki < S; ++ ki) {
      a[:] = sA[ki, threadIdx.y * V : threadIdx.y * V + V];
      b[:] = sA[ki, threadIdx.x * V : threadIdx.x * V + V];
      for (int y = 0; y < V; ++y) {
        for (int x = 0; x < V; ++x) {
          c[y][x] += a[y] * b[x];
        }
      }
    }
  }
  int ybase = blockIdx.y * blockDim.y + threadIdx.y;
  int xbase = blockIdx.x * blockDim.x + threadIdx.x;
  C[ybase * V : ybase*V + V, xbase*V : xbase*V + V] = c[:];
}
```

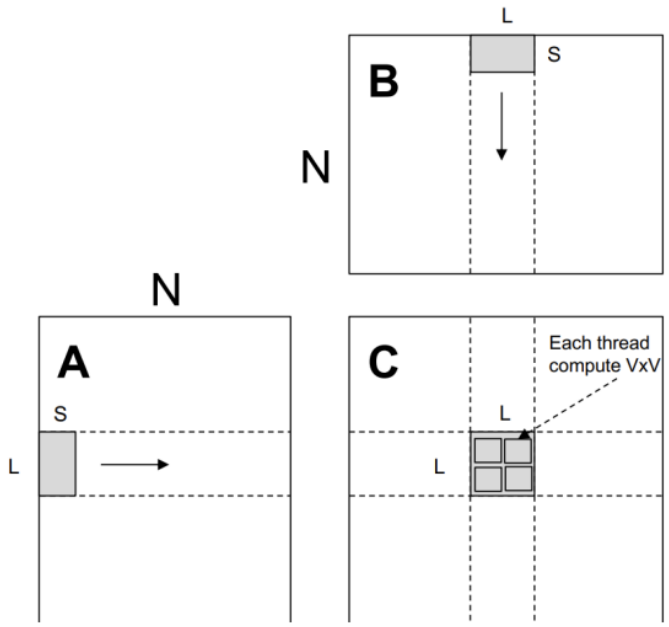
Cooperative Fetching

```
sA[:, :] = A[k : k + S, yblock * L : yblock * L + L];
```

```
int nthreads = blockDim.y * blockDim.x;
int tid = threadIdx.y * blockDim.x + threadIdx.x;

for(int j = 0; j < L * S / nthreads; ++j) {
  int y = (j * nthreads + tid) / L;
  int x = (j * nthreads + tid) % L;
  s[y, x] = A[k + y, yblock * L + x];
}
```

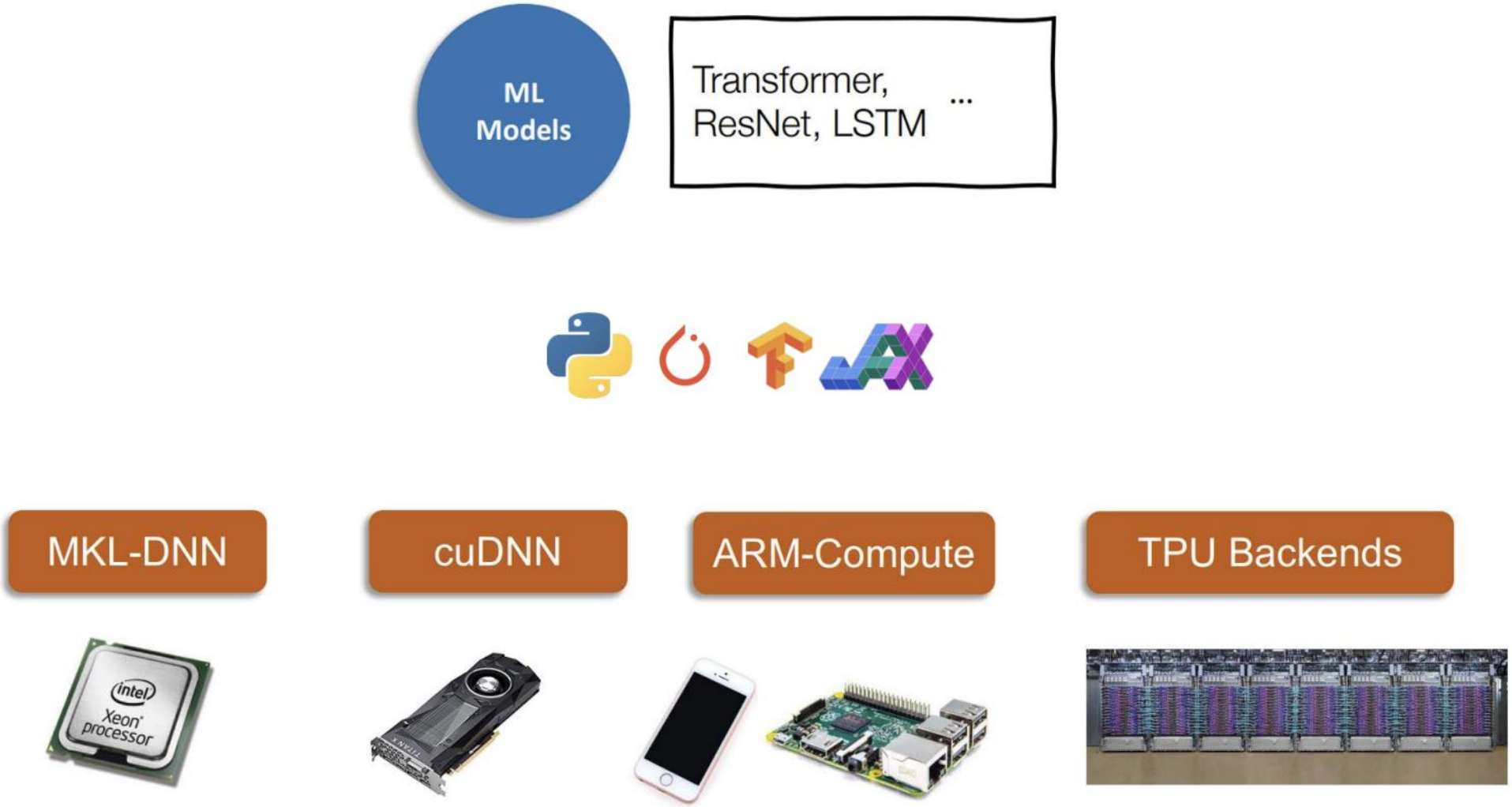
Core Problems Here

- How to choose L/V? Tradeoffs:
  - #threads
  - #registers
  - Amount of SRAM

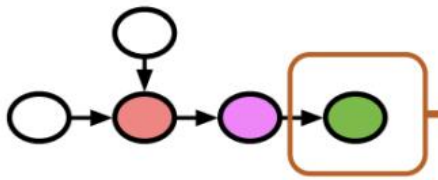



```
__global__ void mm(float A[N][N], float B[N][N], float C[N][N]) {
  __shared__ float sA[S][L], sB[S][L];
  float c[V][V] = {0};
  float a[V], b[V];
  int yblock = blockIdx.y;
  int xblock = blockIdx.x;

  for (int ko = 0; ko < N; ko += S) {
    __syncthreads();
    // needs to be implemented by thread cooperative fetching
    sA[:, :] = A[ko : ko + S, yblock * L : yblock * L + L];
    sB[:, :] = B[ko : ko + S, xblock * L : xblock * L + L];
    __syncthreads();
    for (int ki = 0; ki < S; ++ ki) {
      a[:] = sA[ki, threadIdx.y * V : threadIdx.y * V + V];
      b[:] = sA[ki, threadIdx.x * V : threadIdx.x * V + V];
      for (int y = 0; y < V; ++y) {
        for (int x = 0; x < V; ++x) {
          c[y][x] += a[y] * b[x];
        }
      }
    }
  }
  int ybase = blockIdx.y * blockDim.y + threadIdx.y;
  int xbase = blockIdx.x * blockDim.x + threadIdx.x;
  C[ybase * V : ybase*V + V, xbase*V : xbase*V + V] = c[:];
}
```

In Reality


ML Models
Transformer, ResNet, LSTM ...
MKL-DNN
cuDNN
ARM-Compute
TPU Backends
intel
Xeon processor

Today: Operator Compilation



Specification

```
C = tvm.compute((m, n),
    lambda y, x: tvm.sum(A[k, y] * B[k, x], axis=k))
```

Search Space of Possible Program Optimizations

Low-level Program Variants

```
inp_buffer AL[8][8], BL[8][8]
acc_buffer CL[8][8]
for yo in range(128):
  for xo in range(128):
    vdla.fill_zero(CL)
    for ko in range(128):
      vdla.dma_copy2d(AL, A[ko*8:ko*8+8][yo*8:yo*8+8])
      vdla.dma_copy2d(BL, B[ko*8:ko*8+8][xo*8:xo*8+8])
      vdla.fused_gemm8x8_add(CL, AL, BL)
    vdla.dma_copy2d(C[yo*8:yo*8+8,xo*8:xo*8+8], CL)
```

```
for yo in range(128):
  for xo in range(128):
    C[yo*8:yo*8+8][xo*8:xo*8+8] = 0
    for ko in range(128):
      for yi in range(8):
        for xi in range(8):
          for ki in range(8):
            C[yo*8+yi][xo*8+xi] +=
              A[ko*8+ki][yo*8+yi] * B[ko*8+ki][xo*8+xi]
```

```
for y in range(1024):
  for x in range(1024):
    C[y][x] = 0
    for k in range(1024):
      C[y][x] += A[k][y] * B[k][x]
```

Transforming Loops: Loop Splitting

Code

```
for x in range(128):
  C[x] = A[x] + B[x]
```

```
for xo in range(32):
  for xi in range(4):
    C[xo * 4 + xi]
     = A[xo * 4 + xi] + B[xo * 4 + xi]
```

```
for xi in range(4):
  for xo in range(32):
    C[xo * 4 + xi]
      = A[xo * 4 + xi] + B[xo * 4 + xi]
```

```
def gpu_kernel():
  C[threadId.x * 4 + blockIdx.x] = . . .
```

Triton Programming Model

- Users define tensors in SARM, and modify them using torch-like primitives

Embedded in Python

Kernels are defined in Python using triton.jit

Pointer arithmetics

Users construct tensors of pointers and (de)reference them elementwise

Shape Constraints

Must have power-of-two number of elements along each dimension

Example: elementwise add v1 (z = x + y)

- Triton kernel will be mapped to a single block (SM) of threads
- Users will be responsible for mapping to multiple blocks

```
import triton.language as tl
Import triton

@triton.jit
def _add(z_ptr, x_ptr, y_ptr, N):
  # same as torch.arrange
  offsets = tl.arange(0, 1024)
  # create 1024 pointers to X, Y, Z
  x_ptrs = x_ptr + offsets
  y_ptrs = y_ptr + offsets
  z_ptrs = z_ptr + offsets
  # load 1024 elements of X, Y, Z
  x = tl.load(x_ptrs)
  y = tl.load(y_ptrs)
  # do computations
  z = x + y
  # write-back 1024 elements of X, Y, Z
  tl.store(z_ptrs, z)


N = 1024
x = torch.randn(N, device='cuda’)
y = torch.randn(N, device='cuda’)
z = torch.randn(N, device='cuda’)
grid = (1, )
_add[grid](z, x, y, N)
```

Example: elementwise add v2 (z = x + y)

Use multiple blocks

- Index the block and apply offset
- Adds bound check

```
import triton.language as tl
Import triton

@triton.jit
def _add(z_ptr, x_ptr, y_ptr, N):
  # same as torch.arrange
  offsets = tl.arange(0, 1024)
  offsets += tl.program_id(0)*1024
  # create 1024 pointers to X, Y, Z
  x_ptrs = x_ptr + offsets
  y_ptrs = y_ptr + offsets
  z_ptrs = z_ptr + offsets
  # load 1024 elements of X, Y, Z
  x = tl.load(x_ptrs, mask=offset<N)
  y = tl.load(y_ptrs, mask=offset<N)
  # do computations
  z = x + y
  # write-back 1024 elements of X, Y, Z
  tl.store(z_ptrs, z)


N = 192311
x = torch.randn(N, device='cuda’)
y = torch.randn(N, device='cuda’)
z = torch.randn(N, device='cuda’)
grid = (triton.cdiv(N, 1024), )
_add[grid](z, x, y, N)
```

Listing 5. $A = \max(A, 0)$ in Triton-IR. Note that tile shapes are non-parametric here. In this paper their values are instantiated by the Triton-JIT.

```
define kernel void @relu(float* %A, i32 %M, i32 %N) {
prologue:
  %rm = call i32<8> get_global_range(0);
  %rn = call i32<8> get_global_range(1);
  ; broadcast shapes
  %1 = reshape i32<8, 8> %M;
  %M0 = broadcast i32<8, 8> %1;
  %2 = reshape i32<8, 8> %N;
  %N0 = broadcast i32<8, 8> %2;
  ; broadcast global ranges
  %3 = reshape i32<8, 1> %rm;
  %rm_bc = broadcast i32<8, 8> %3;
  %4 = reshape i32<1, 8> %rn;
  %rn_bc = broadcast i32<8, 8> %4;
  ; compute mask
  %pm = icmp slt %rm_bc, %M0;
  %pn = icmp slt %rn_bc, %N0;
  %msk = and %pm, %pn;
  ; compute pointer
  %A0 = splat float*<8, 8> %A;
  %5 = getelementptr %A0, %rm_bc;
  %6 = mul %rn_bc, %M0;
  %pa = getelementptr %5, %6;
  ; compute result
  %a = load %pa;
  %_0 = splat float<8, 8> 0;
  %result = max %float %a, %_0;
  ; write back
  store fp32<8, 8> %pa, %result
}
```

5.1.1 Pre-Fetching

Tile-level memory operation inside loops can be problematic, as they may induce severe latency that cannot be hidden in the absence of enough independent instructions. It is however possible to mitigate this problem in Triton-IR directly by detecting loops and adding adequate prefetching code where necessary (See Listing 7).

Listing 7. Automatic pre-fetching

```
B0:
 %p0 = getelementptr %1, %2
B1:
 %p = phi [%p0,B0], [%p1,B1]
 %x = load %p
 ; increment pointer
 %p1 = getelementptr %p, %3
```

```
B0:
 %p0 = getelementptr %1, %2
 %x0 = load %p0
B1:
 %p = phi [%p0,B0], [%p1,B1]
 %x = phi [%x0,B0], [%x1,B1]
 ; increment pointer
 %p1 = getelementptr %p, %3
 ; prefetching
 %x1 = load %p
```

5.1.2 Tile-Level Peephole Optimization

The presence of tile-level operations in Triton-IR offers new opportunities for peephole [29] optimizers. For instance, chains of transpositions can be simplified using the identity $X = (X^T)^T$ for any tile X. We believe that other algebraic properties related to e.g., diagonal tiles could also be exploited in the future.

5.2.1 Hierarchical Tiling

Nested tiling strategies (see Figure 5) aim at decomposing tiles into micro-tiles and eventually nano-tiles in order to fit a machine's compute capabilities and memory hierarchy as tightly as possible. While this technique is routinely used in auto-tuning frameworks [34, 40], the structure of Triton-IR makes it possible to automatically enumerate and optimize valid nested tiling configurations for any expressible program (and without the need for polyhedral machinery).

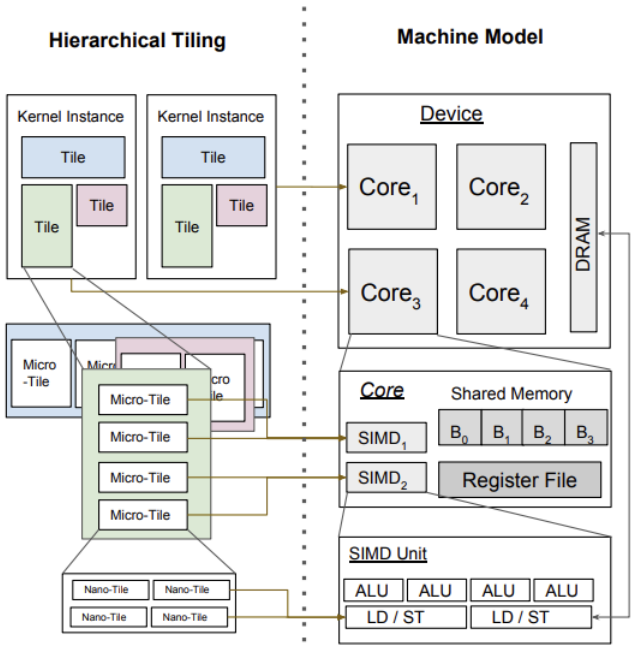


Figure 5. Hierarchical Tiling in the Triton-IR Machine Model

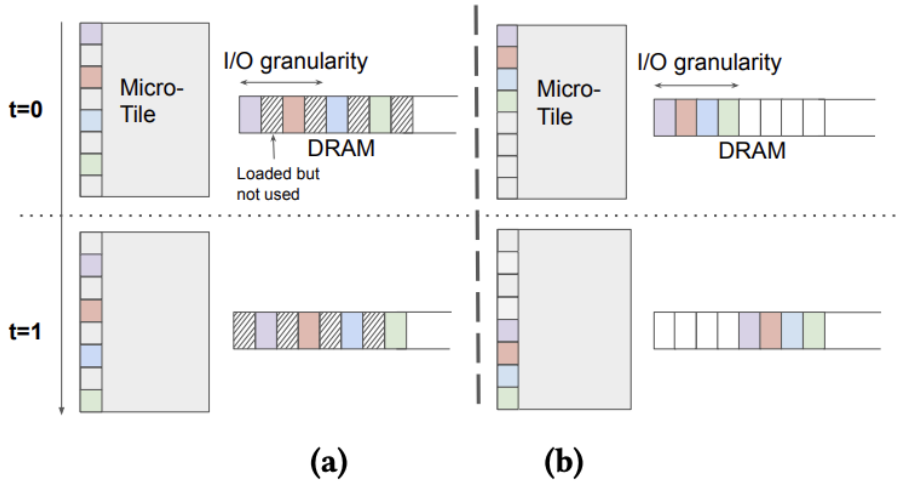


Figure 6. Uncoalesced (a) and coalesced (b) DRAM accesses. Different threads are shown in different colors.

5.2.3 Shared Memory Allocation

Tile-level operations that have high arithmetic intensity (e.g., `dot`) can benefit from temporarily storing their operands in fast shared memory. The purpose of the Shared Memory Allocation pass is to determine when and where a tile should be stashed to this space. This can be done, as illustrated in Figure 7, by first calculating the *live range* of each variable of interest, and then using the linear-time storage allocation algorithm proposed in [15].

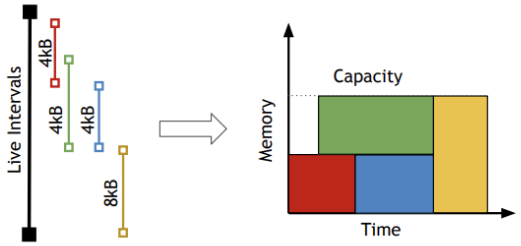
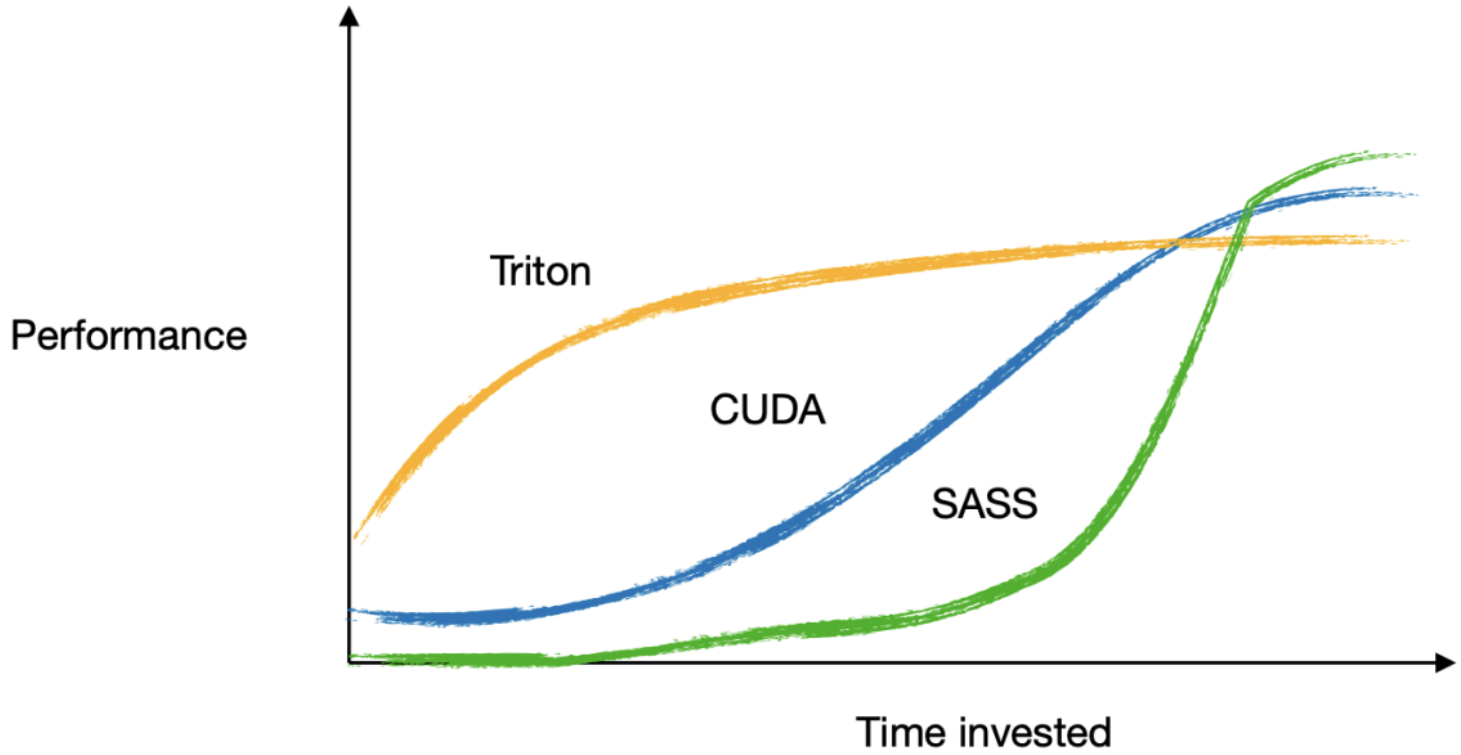


Figure 7. Shared Memory Allocation

Why Triton (seemingly) Succeeds



SASS = streaming assembly